Easz: An Agile Transformer-based Image Compression Framework for Resource-constrained IoTs

Yu Mao*, Jingzong Li†, Jun Wang‡, Hong Xu§, Tei-Wei Kuo¶, Nan Guan‡, Chun Jason Xue*
*Mohamed bin Zayed University of Artificial Intelligence, UAE, {yu.mao, jason.xue}@mbzuai.ac.ae
†The Hang Seng University of Hong Kong, Hong Kong, jingzongli@hsu.edu.hk
‡City University of Hong Kong, Hong Kong, {jwang699-c, nanguan}@my.cityu.edu.hk
§The Chinese University of Hong Kong, Hong Kong, hongxu@cuhk.edu.hk
¶National Taiwan University, Taiwan, ktw@csie.ntu.edu.tw

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132645

***Abstract*—Neural image compression, necessary in various machine-to-machine communication scenarios, suffers from its heavy encode-decode structures and inflexibility in switching between different compression levels. Consequently, it raises significant challenges in applying the neural image compression to edge devices that are developed for powerful servers with high computational and storage capacities. We take a step to solve the challenges by proposing a new transformer-based edge-compute-free image coding framework called Easz. Easz shifts the computational overhead to the server, and hence avoids the heavy encoding and model switching overhead on the edge. Easz utilizes a patch-erase algorithm to selectively remove image contents using a conditional uniform-based sampler. The erased pixels are reconstructed on the receiver side through a transformer-based framework. To further reduce the computational overhead on the receiver, we then introduce a lightweight transformer-based reconstruction structure to reduce the reconstruction load on the receiver side. Extensive evaluations conducted on a real-world testbed demonstrate multiple advantages of Easz over existing compression approaches, in terms of adaptability to different compression levels, computational efficiency, and image reconstruction quality.**

***Index Terms*—Image Compression, Erase-and-Squeeze, Transformer-based Auto-Encoder.**

I. Introduction

The need for advanced lossy image compression is raised by the explosive development of edge devices equipped with high-resolution cameras, such as industrial-inspections system [8], wildlife observation system [5], and autonomous driving [31], by analyzing massive data sensed by heterogeneously connected devices. Transmitting the huge volume of data generated by IoT devices can cause significant channel congestion, particularly in scenarios with limited bandwidth availability [11]. Neural-network (NN) based compressor can provide a better compression performance and outperform traditional image compression techniques like JPEG [10] and BPG [3]. However, due to its heavy, symmetric encoding and decoding structure and inflexible compression rate adjustment, current NN-based methods have not yielded practical use on resource-constrained edge devices.

Current edge NN-compression/decompression latency is unsatisfied due to the paucity of computational ability and storage on edge devices [25, 1]. As shown in Fig. 1, encoding an small 512×768 image can take as long as 18015 ms on high-end devices equipped with GPU like the NVIDIA Jetson TX2, not to mention current camera usually capture images to 4K size. Switching compression level can also cause $286 \sim 11600$ ms overhead, far exceeding the transmission time $151 \sim 163$ ms.

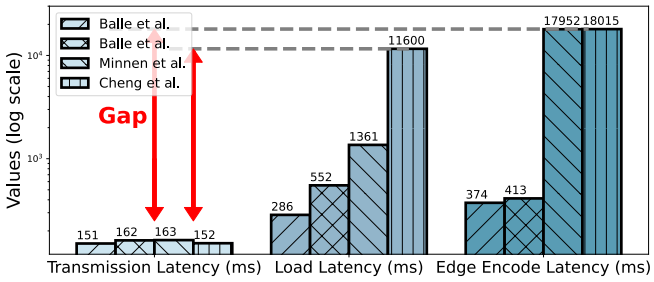


Fig. 1: NN-based compressors face challenges on edge devices like the Jetson TX2, where loading and encoding an image can take over 10 seconds compared to a transmission latency of about 0.1 seconds.

In this paper, we introduce Easz, an agile image coding framework that operates efficiently at both the edge and server side. To achieve agile encoding, Easz includes an erase-and-squeeze process, which removes the image patches based on a conditional random-based sampler. This technique can provide fine-grained compression level choices with fast adaptation to various scenarios. To improve the reconstruction quality, we introduce the novel transformer architecture to conduct fine-grained pixel-level reconstruction. However, the pixel-level reconstruction using naive transformer architecture is computationally intensive ($O(65536^2 \times 768)$ for a 256×256 image), even for a powerful 2080 Nvidia GPU server. We then propose a lightweight transformer architecture for efficient, high-quality reconstruction of erased patches on the receiver side. This involves a two-stage image patchify process to limit the scope of attention correlation calculations and a four-layer light weight transformer model for pixel-level local image

reconstruction.

The key contributions of this work can be summarized as:

- Generalized Erase-and-Squeeze Process: A new paradigm is introduced that offers more refined and flexible image reduction ratios;
- Receiver-Side Lightweight Transformer Architecture: A super-lightweight (8.7MB) transformer architecture is designed for efficient and high-quality reconstruction of erased patches;
- Compatibility with Existing Algorithms: Easz is compatible with all existing image compression algorithms and can also function independently;
- We conduct extensive experiments that verify the effectiveness of Easz.

II. Related Work

Image compression is experiencing significant growth [28, 29, 19, 6, 16]. Notable developments include the introduction of auto-regressive components [19], Gaussian Mixture Models for probability estimation [6], and general-purpose lossless compressiors using lightweight neural networks [16, 15, 18].

Despite progress, real-world applications still face challenges at the edge [14, 21, 11]. [21] leverages in-sensor compression and selective ROI prioritization to optimize high-resolution image processing for edge ML. Deep-learning-based compression methods take about 1$\sim$20 second per image (512×768) on NVIDIA Jetson TX2, and many real-life endpoints are less potent than the TX2 (considering Raspberry Pi 4) but still need to compress images. A primary issue is that most NN-based image compressors require a model switch when changing compression levels. [11] uses semantic-driven compressive sensing to enhance image compression for resource-constrained IoT systems. Another approach involves downsampling images at the edge and using super-resolution techniques to reconstruct them on the server [30]. These methods reduce computational load at the edge, but applying super-resolution directly in this context results in an inflexible downsizing rate and can degrade reconstruction performance [12].

III. System Design

The paper presents Easz, a novel edge-optimized image compression framework. It merges erase-based compression on the edge with a lightweight transformer-powered reconstruction on the server side, outperforming conventional codecs like JPEG and neural-network-driven compressors.

A. Erase-and-Squeeze Algorithm

The section outlines the proposed *Erase-and-Squeeze* strategy, including erase mask generation and the organization of un-erased image components on the edge/sender side.

Erase mask generation. An erasing mask is a binary matrix used to determine if subpatches in an image patch should be kept or erased, defined as $\mathbf{M}$ where each entry is set as follows:

$$\mathbf{M}[i,j] = 1 \text{ if } (i,j) \text{ is sampled, else } 0$$

Fig. 2 illustrates some mask examples with white blocks representing erased subpatches. As shown in Fig. 2(b), a simple approach to erase is to remove blocks along the diagonal, which allows for easy reorganization of the remaining sections. However, the diagonal mask is not generalized due to its fixed erase ratio. Super-resolution-based approaches also suffer from a fixed file size reduction ratio due to uniform down-sampling.

To achieve flexible size reduction, a naive solution is to randomly erase a portion of patches. However, this can cause significant distortion due to consecutive information loss. Fig. 2(a) shows a randomly generated mask that create large continuous erased areas, causing performance degradation in subsequent JPEG and reconstruction stages (See Fig. 3).

We then propose a generalized paradigm, a row-based conditional sampler for generating erase masks, covering various types including diagonal and uniform masks. This approach allows for a highly flexible sampling rate, while only need one model to reconstruct at any sampling rate. Also, it has better reconstruction behavior compared with a random masks.

Row-based Sampler Definition. In this sampler, each row i of the image is processed sequentially, and within each row, the column coordinate j is selected using a sampler. Thus, the Row-based Sampler is defined by two functions, g_r^1, which governs the random column selection from $\mathbf{X}$, while $g_r^0(i)$ represents the current row:

$$\hat{\mathbf{X}}[i,j] = \mathbf{X}[g_r^0(i), g_r^1(i,j)]$$

where $g_r^0(i)$ is a deterministic function representing row selection, and $g_r^1(i,j)$ is a random mapping for the column coordinate within row i.

Constraints for Row-based Sampler. When sampling from a matrix $X \in \mathbb{R}^{H \times W}$, where each row is sampled T times, the new sample $x_{i,t+1}$ is subject to:

1. Intra-row constraint (avoid proximity to previous samples in the same row):

$$\begin{aligned} &g_r^1(i,t+1) \sim \text{Uniform}(0, W-1) \\ &\text{s.t.} \quad \left| g_r^1(i,t+1) - g_r^1(i,t) \right| > \delta \end{aligned} \tag{1}$$

Here, δ is a threshold distance that ensures the newly sampled column $g_r^1(i,t+1)$ is sufficiently distant from the previously selected columns $\{g_r^1(i,0), \ldots, g_r^1(i,t)\}$.

2. Inter-row constraint (minimize adjacency to prior samples from the preceding row):

$$\begin{aligned} &g_r^1(i,t+1) \sim \text{Uniform}(0, W-1) \\ &\text{s.t.} \quad \left| g_r^1(i,t+1) - g_r^1(i-1,T) \right| > \Delta \end{aligned}$$

Similarly, Δ represents a minimum separation between the newly sampled column in row i and the previously selected columns $\{g_r^1(i-1,0), \ldots, g_r^1(i-1,T)\}$ from row $i-1$.

Under these constraints, the row-based random sampler can be formalized as:

$$G_r = \left\{ g_r^0(i) = I, g_r^1(i,t+1) \right\},$$

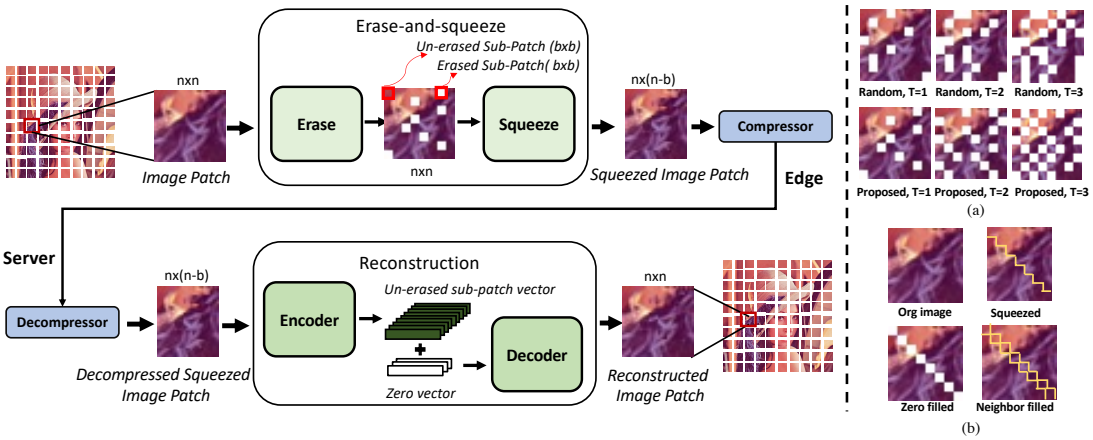
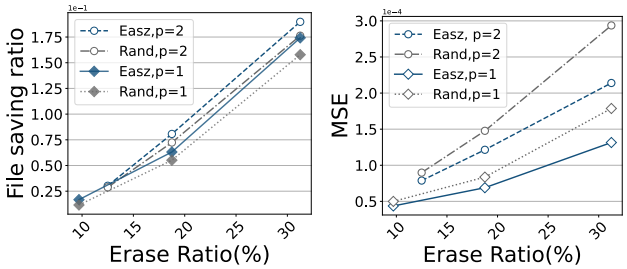


Fig. 2: Left: Easz system overview. A image is going through a two-stage image patchify process, erase-and-squezee process. The squeezed image is then compressed using existing compressor and transmit. On the decompression stage, the erased patch is recovered. Right: (a) Proposed erase methods compared with random erase methods. T indicates an erased item in each row. (b) Different methods to re-organize un-erased image components.



(a) Impact on JPEG (↑ better). (b) Impact on Recon (↓ better).

Fig. 3: The proposed method outperforms random masking in terms of JPEG impact and reconstruction, resulting in a higher file saving ratio and lower MSE on Kodak dataset. The variable p represents patch size.

Fig. 4: Easz preserves details better than SR methods via direct pixel prediction, improving PSNR and SSIM.

where

$$g_r^1(i,t+1) \sim \text{clip}\left(\text{Uniform}(0, W-1)\right)$$
$$\text{s.t.} \quad |g_r^1(i,t+1) - g_r^1(i,t)| > \delta$$
$$|g_r^1(i,t+1) - g_r^1(i-1,T)| > \Delta$$

When restricted to T=1 with non-adjacent sampling in both row and columns, it becomes a diagonal mask. With patch=1, T=n/2, and non-adjacent sampling in each row and column, it degrades to 2x super-resolution. Also, we will show in experiment part that generating patches directly results in better reconstruction than existing super-resolution methods.

Squeeze. After applying the generated erase mask $\mathbf{M}$ on $\mathbf{X}$, the next step is to squeeze the non-zero (sampled) locations together to form a smaller image $\mathbf{X}_{\text{squeezed}} \in \mathbb{R}^{h' \times w' \times C}$, where $h' < h$ and $w' < w$ represent the dimensions. This can be achieved by filtering out the zero entries from $\mathbf{X}$:

$$\mathbf{X}_{\text{squeezed}}[i', j'] = \mathbf{X}[i, j] \quad \text{for all } (i, j) \text{ where } \mathbf{M}[i, j] = 1$$

After the process, the squeezed image $\mathbf{X}_{\text{squeezed}}$ would be encoded using compressors like JPEG, BPG, etc to get a compressed form $\hat{\mathbf{X}}_{\text{squeezed}}$ and send, as illustrated in Fig. 2.

B. Reconstruction

Recent work explores pixel-level generation for high-fidelity reconstruction [13]. Easz adopts a transformer to perform pixel-level reconstruction, achieving better results than super-resolution methods, as shown in Fig. 4. It also supports flexible downsampling ratios for adaptive compression. However, directly apply transformers to predict pixels is costly due to the nature of attention mechanisms. In our experiments, a 12GB Nvidia 2080Ti server cannot afford to run pixel-based prediction of a single 256×256 image. Rather than applying conventional model compression techniques [17], our approach is inspired by the block-based strategies employed in traditional image compressors such as JPEG, which use 32×32 or 64×64 blocks to enhance compression performance. We design a two-stage image patchification process followed by a lightweight (8.4MB) transformer for sub-patch restoration.

Complexity Analysis. Predicting pixel values for each element in the image, particularly in high-resolution images, proves to be computationally expensive. Consider a grayscale

image of size 256x256 with a 1x1 patch size. This configuration results in 65,536 pixels. Based on the attention complexity, an image of size (h, w) in grayscale would require $O((hw)^2 \times d_{model})$ calculations if using a pixel as a token. Therefore, calculating a single self-attention for this setup would necessitate $65536^2 \times d_{model} = 4,294,967,296 \times d_{model}$ calculations. Such a computational load is heavy to execute on today's hardware, and it becomes exponentially more challenging when dealing with higher-resolution images.

Two-Stage Image Patchify. In the Easz framework, a two-stage image patchify process is employed to reduce the computational complexity of the vision transformer. Given an input image X with dimensions (h, w), it is first divided into non-overlapping image patches of size (n, n). Each of these $n \times n$ image patches is further subdivided into smaller non-overlapping (b, b) sub-patches. Critical operations such as erasure, squeezing, and reconstruction are performed at the sub-patch level. This patchify process is essential to manage the computational challenge of the vision transformer.

Next, we will explain how the proposed two-stage image patchify process reduces attention complexity. For clarity, the following analysis omit d_{model} as it remains constant. The first stage patchify process split the image into $\frac{hw}{n^2}$ patches, with $n \times n$ size. The computational complexity on this stage is $O(\frac{(hw)^2}{n^4})$. Then, the second patchify process is executed, split the $n \times n$ patches into $b \times b$ sub-patches. Therefore there will be $\frac{(hw)^2}{n^2} \times \frac{n^2}{b^2}$ sub-patches, with $b \times b$ size. We configure the attention mechanism to operate within each image patch, thereby constraining the computational complexity to scale with the size of patches rather than the entire image, resulting in a computational complexity $O(\frac{hw}{n^2} \times \frac{n^4}{b^4} = O(\frac{(hw \times n^2)}{b^4})$. Therefore even when the token size is small, like when $b = 1$, the calculation complexity is still $O(hwn^2)$, which is much smaller than the original complexity $O(hw)^2)$.

Now recall that image of size $(256, 256)$, which would cost $4,294,967,296 \times d_{model}$ calculations originally, after the two-stage image patchify process where $n = 32$ and $b = 4$, would require $1,048,576 \times d_{model}$ calculations, reduced by 4096 times compared to the original.

Model Structure. Fig. 5 illustrates the architecture of our efficient transformer-based reconstruction network. The encoder and decoder are composed of two transformer blocks, each containing three layernorms, one attention layer, and one feedforward layer. Note that the model can work with a variety of input erase ratios, which are controlled by k and b, and hence, we do not need to train a specific model for each erase ratio. Sub-patches can execute in parallel for Easz due to the nature of transformer block calculations [26].

Training Process. Consider a batch of image patches $X_1, X_2, \ldots, X_n$ (each $n \times n$) randomly chosen from our training dataset, with patchification already completed. These patches are divided into smaller sub-patches of size $(b \times b)$, resulting in $(n/b, n/b)$ sub-patches per original patch. Positional embeddings are multiplied with these sub-patches to provide spatial context. An erase mask is then applied to remove k random sub-patches. The encoder E receives the remaining un-erased sub-patches $U = \{u_1, u_2, \ldots, u_m\}$ where $m = n/b \times n/b - k$. Each un-erased sub-patch u_i is projected into an embedding e_i. The two-layer encoder processes these embeddings to extract features: $F = E(e_1, e_2, \ldots, e_m)$. To compensate for erased blocks during reconstruction, we introduce zero vectors $\hat{F} = \{\hat{f}_1, \hat{f}_2, \ldots, \hat{f}_k\}$. We align $\hat{F}$ with the zero-value positions and F with one-value positions based on stored erase mask information. The combined feature sets $\{F, \hat{F}\}$ are fed into a two-layer decoder D which reconstructs the image: $\hat{X} = D(\{F, \hat{F}\})$. Our goal is to fine-tune encoders and decoders (E and D, respectively) so that they minimize discrepancies between original images (P) and their reconstructions ($\hat{X}$), despite interference from zero vectors.

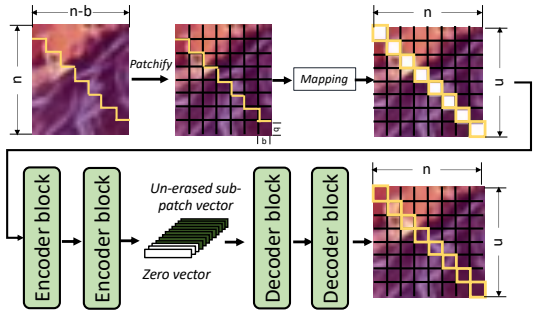


Fig. 5: Reconstruct process illustration.

To minimize the difference between the original image X and the reconstructed image $\hat{X}$, we adopt LPIPS [32], a well-known perceptual loss, along with L1 loss as training loss.

$$\text{Loss}(x, y) = \text{L1}(x, y) + \lambda * \text{LPIPS}(x, y) \tag{2}$$

The feature extraction model is selected as VGG and λ is chosen as 0.3 in our experiments.

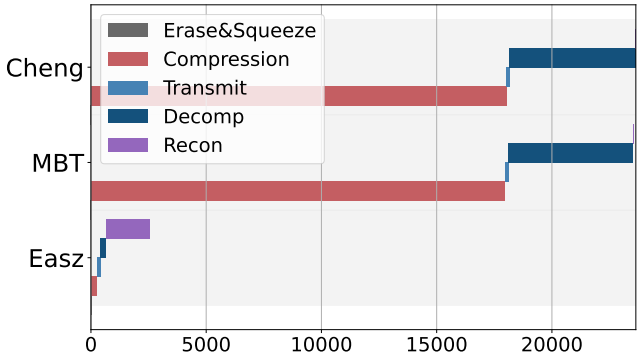
IV. Experiments

A. Experimental Setting

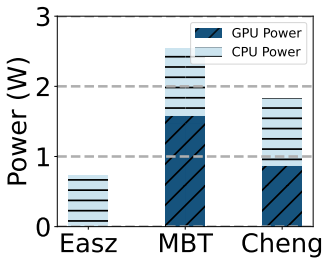
Training setting. The experiments consist of two phases: offline pretraining and online testing. In the pretraining phase, a specific loss function (Eq. 2) is used with these hyperparameters: learning rate of 2.8e-4, erase ratio of 0.25, batch size of 4096, weight decay of 0.05. Randomly generated erase masks are applied for model robustness during this stage. For online testing, a consistent mask is utilized on both edge and server sides. But note that transmission of the mask won't be costly due to its small size—a binary mask at dimensions 32×32 occupies only 128 bytes.

Hardware platforms. Our framework is implemented with ~1000 lines of Python. We use an NVIDIA Jetson TX2 as the edge device and a desktop with Intel i7-9700K CPU and RTX 2080Ti GPU as the server, which are physically connected to a Wi-Fi router and communicate via TCP.

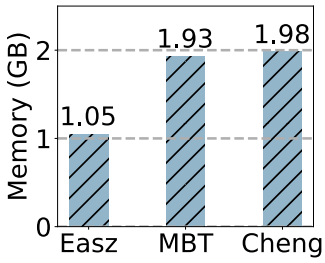
Datasets. During the offline pretraining phase, the CIFAR-10 [2] dataset is employed to pretrain the model, enabling



(a) End-to-End Latency Breakdown(ms).



(b) Encode Power consumption.



(c) Encode Memory footprint.

Fig. 6: Efficiency Evaluation on NVIDIA Jetson TX2.

it to acquire generative capabilities. In the testing phase, two common image compression datasets, Kodak [7] and CLIC [27], are utilized to assess the generative performance of the proposed method.

Metrics. To benchmark against other super-resolution approaches, we include PSNR and SSIM to demonstrate the superiority of our method. Since removing content would generally impact reference-based metrics such as PSNR and SSIM (a trend also observed in other downsampled-and-super-resolution methods), we employ non-reference perceptual metrics for comparison with other compression baselines: Brisque [20], Pi [4], and Tres [9]. Compression performance is evaluated by bits per pixel (BPP).

Baselines. We use four compression methods as baselines to demonstrate the effectiveness of the proposed method: JPEG, BPG, MBT[19], and Cheng-Anchor [6]. Among these, MBT and Cheng-Anchor are two NN-based compression methods.

B. Latency Analysis and Resource Consumption

We first report the latency breakdown of Easz with other neural network-based compression methods, using a Jetson TX2 for compression and a server for decompression. We repeat the runs 24 times and report the average on Fig. 6a. We observe that Erase-and-Squeeze only takes up 0.7% of the end-to-end latency, which induces minimal overhead on the edge device and proves Easz' efficiency, while both MBT and Cheng-Anchor are too compute-intensive to run the compression on the edge side. As expected, the reconstruction in Easz takes the longest time, accounting for 74% of the latency. We argue that the performance can be significantly improved by upgrading to a datacenter-class GPU such as the A100. Additionally, acceleration using FPGAs presents an interesting direction for future exploration [22, 23].

Resource consumption is another critical consideration when it comes to resource-constrained edge devices. To assess this, we measure three key metrics – CPU power, GPU power, and memory footprint – using the Tegrastats Utility [24] on the Jetson TX2. As illustrated in Fig. 6, our findings reveal that Easz excels in all metrics compared to other NN-based compression methods. Specifically, in contrast to MBT and Cheng-Anchor, Easz achieves a remarkable 71.3% and 59.9% reduction in total power consumption. It's noteworthy that Easz does not utilize any GPU power on the edge device, attributed to its lightweight yet effective erase-and-squeeze design. Furthermore, Easz reduces memory footprint by 45.8% and 47.1%, respectively. These results underscore the advantage of deploying Easz on wimpy edge devices.

TABLE I: Comparison with Super-Resolution on Kodak Dataset.

Metrics	Easz	SwinIR	realESRGAN	BSRGAN
PSNR	28.96	24.86	24.85	25.35
MS_SSIM	0.96	0.94	0.93	0.94
Recon Model Size	8.7MB	67MB	67MB	67MB

C. Comparison with Super-Resolution Methods

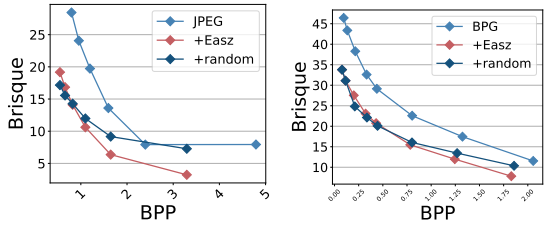
We compare the reconstruction effect of Easz with state-of-the-art super-resolution methods to demonstrate Easz's effectiveness. As shown in Tab. I, Easz outperforms super-resolution in pixel-level reconstruction metrics while having a much more flexible reduction ability. Note that Easz uses a model of only 8.7MB, while other models are 67MB.

D. Ablation Study

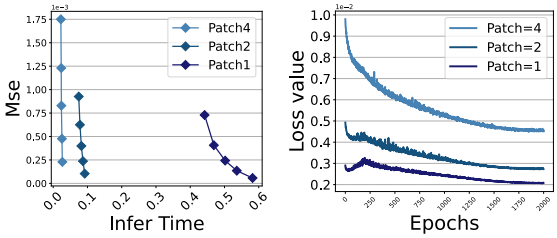
Effectiveness of erase strategy. Fig. 7a and Fig. 7b compare the proposed to erase mask generation method, the random erase mask method, and the baseline (JPEG and BPG) throughout the entire pipeline. It can be observed that the proposed erase mask generation method achieves better BPP at the same quality level on both JPEG and BPG, further substantiating the effectiveness of the proposed method.

Patch size selection. Fig. 7c examines the effects of two hyperparameters, erase block size (1, 2, and 4) and erase ratio (10% to 50%), on compression rate and quality. As the erase ratio increases, MSE rises, indicating lower reconstruction quality. Smaller patch sizes yield better reconstruction due to higher local correlations. Patch size=2 offers a balance between speed—being six times faster than size=1—and quality—with only a slight difference in MSE. Doubling the patch size from 2 to 4 also doubles both speed and MSE. The recommendation is to use smaller patch sizes for practical applications but consider size=2 for additional speed needs.

Effectiveness of fine-tuning. Our model, after pretraining on the CIFAR dataset for 5000 epochs, can be applied to various image compression tasks due to its ability to recognize similarities in local image features. Typically, models are first



(a) JPEG, Brisque(↓ better). (b) BPG, Brisque(↓ better).



(c) MSE(↓ better). (d) MSE(↓ better)

Fig. 7: (a)(b): Comparison between Easz with proposed mask strategy, Easz with random mask strategy, and conventional compression baselines (JPEG and BPG). (c) Patch size and erase ratio's impact on MSE. (d) MSE during Easz fine-tuning process with patch size=1,2,4 on Kodak dataset.

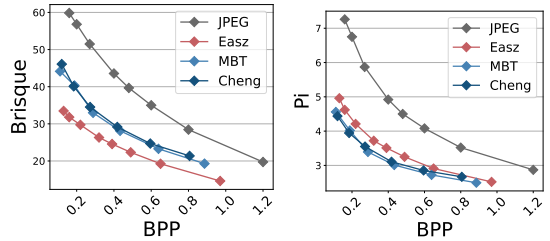
TABLE II: Compression Performance Enhancement on Kodak Dataset and Clic Dataset.

Metrics		JPEG		BPG		MBT		Cheng-anchor	
		Org	*+Proposed*	*Org*	*+Proposed*	*Org*	*+Proposed*	*Org*	*+Proposed*
Kodak	BPP	0.412	0.411	0.382	0.410	0.433	0.389	0.418	0.402
	Brisque	43.06	22.34	30.675	23.27	28.13	18.63	29.16	20.51
	Pi	4.84	3.33	3.07	3.04	3.01	3.00	3.11	3.05
	Tres	77.62	86.26	83.55	85.88	84.14	88.03	88.53	89.80
Clic	BPP	0.306	0.307	0.308	0.293	0.308	0.292	0.287	0.267
	Brisque	60.51	23.63	39.95	25.27	32.20	18.37	35.42	21.55
	Pi	8.51	5.02	4.85	4.66	4.33	4.35	4.58	4.50
	Tres	50.65	63.69	65.14	67.08	73.54	78.30	82.91	83.95

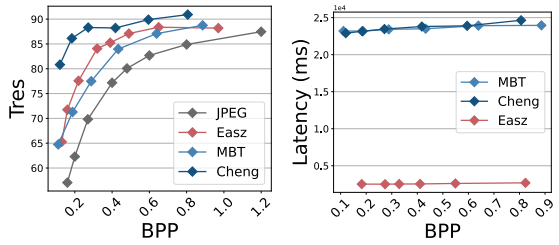
pre-trained on a large dataset and then fine-tuned for specific tasks. We tested if fine-tuning our pretrained model with the Kodak dataset would be beneficial and found that it indeed improves performance by reducing losses across different patch sizes (1×1, 2×2, and 4×4), as shown in Fig. 7d.

E. Improvement on Existing Compressors

To evaluate how well Easz works with leading compressors, we incorporated it into four established methods: JPEG and BPG (traditional compressors), as well as MBT and Cheng-anchor (neural network-based compressors). We used two datasets, Kodak and CLIC, to test the resilience of Easz across different types of image data. For the Kodak dataset, we aimed for a bit-per-pixel (BPP) rate of approximately 0.4; for the CLIC dataset, we targeted a BPP of around 0.3 to gauge Easz's efficacy at varying levels of compression. The results showing how each baseline method performs on its own and when combined with Easz are detailed in Tables II. The data clearly shows that integrating these baselines with Easz consistently enhances perceptual quality without increasing BPP.



(a) Brisque (↓ better). (b) Pi (↓ better).



(c) Tres (↑ better). (d) End-to-end Latency.

Fig. 8: Compression performance of Easz, JPEG, MBT and Cheng on three perceptual metrics (a-c). Fig. 8d evaluates the end-to-end latency on our testbed.

F. End-to-End Compression Performance

In this experiment, we use JPEG+Easz as the baseline and observe changes in three perceptual metrics at different bitrates (BPP). Notably, JPEG alone underperforms compared to two deep-learning compression methods at all compression levels. However, with Easz enhancement, JPEG shows a marked improvement. For the BRISQUE metric specifically, JPEG+Easz exceeds both deep-learning methods. Regarding the Pi metric, JPEG+Easz matches the performance of these methods. With the Tres metric, while JPEG+Easz outdoes MBT, it falls short of Cheng-anchor's results. Overall, Easz boosts JPEG to compete effectively with other state-of-the-art deep-learning compression techniques in each perceptual measure. Easz also outperforms two neural network-based methods in latency, with an average end-to-end latency of 2568ms across different bitrates per pixel, marking an 89% reduction compared to MBT and Cheng's methods.

V. Conclusion

This paper proposes Easz, which overcomes the challenges faced by modern neural image compression on edge devices. Easz is a comprehensive and efficient data compression framework, which shifts the computational burden to the server side. Easz includes an Erase-And-Squeeze process on the edge, coupled with a transformer-based encoder-decoder architecture on the server. Through careful design of the Erase-and-Squeeze strategy, Easz enhances the performance of existing methods while enabling dynamic adjustment of compression levels. As a result, Easz eliminates the computational and storage burdens on the edge while simultaneously boosting compression rates and transmission efficiency. Our real-world evaluation in an edge-server testbed demonstrates Easz's improvement.

References

[1] *AI and Compute*. 2018. URL: https://openai.com/blog/ai-and-compute/.

[2] Krizhevsky Alex et al. "Learning multiple layers of features from tiny images". In: *2009* (2009).

[3] Fabrice Bellard. *BPG*. https://bellard.org/bpg/. 2014.

[4] Yochai Blau et al. "The 2018 PIRM challenge on perceptual image super-resolution". In: *ECCV Workshops*. 2018.

[5] *Bringing Cutting-Edge Technology to Wildlife Conservation*. https://www.wildlifeinsights.org/. 2023.

[6] Zhengxue Cheng et al. "Learned Image Compression with Discretized Gaussian Mixture Likelihoods and Attention Modules". In: *CVPR*. 2020.

[7] Eastman Kodak Company. *Kodak Lossless True Color Image Suite*. https://r0k.us/graphics/kodak/. 1993.

[8] Shilpa George et al. "Towards Drone-Sourced Live Video Analytics for the Construction Industry". In: *Proceedings of the 20th International Workshop on Mobile Computing Systems and Applications*. ACM. 2019, pp. 3–8.

[9] S Alireza Golestaneh et al. "No-reference image quality assessment via transformers, relative ranking and self-consistency". In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2022, pp. 1220–1230.

[10] JPEG Group. *JPEG*. https://jpeg.org/. 1986.

[11] Yujun Huang et al. "Compressive sensing based asymmetric semantic image compression for resource-constrained IoT system". In: *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 2022, pp. 877–882.

[12] Charles Laroche et al. "Deep model-based super-resolution with non-uniform blur". In: *Proceedings of the IEEE/CVF winter conference on applications of computer vision*. 2023, pp. 1797–1808.

[13] Zhenglin Li et al. "Mapping new realities: Ground truth image creation with pix2pix image-to-image translation". In: *arXiv preprint arXiv:2404.19265* (2024).

[14] Yu Liang et al. *Ariadne: A Hotness-Aware and Size-Adaptive Compressed Swap Technique for Fast Application Relaunch and Reduced CPU Usage on Mobile Devices*. 2025. arXiv: 2502.12826 [cs.OS]. URL: https://arxiv.org/abs/2502.12826.

[15] Yu Mao et al. "Accelerating General-Purpose Lossless Compression via Simple and Scalable Parameterization". In: *Proceedings of the 30th ACM International Conference on Multimedia*. 2022, pp. 3205–3213.

[16] Yu Mao et al. "Faster and Stronger Lossless Compression with Optimized Autoregressive Framework". In: *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 2023, pp. 1–6.

[17] Yu Mao et al. "On the compressibility of quantized large language models". In: *arXiv preprint arXiv:2403.01384* (2024).

[18] Yu Mao et al. "TRACE: A Fast Transformer-based General-Purpose Lossless Compressor". In: *Proceedings of the ACM Web Conference 2022*. 2022, pp. 1829–1838.

[19] David Minnen et al. "Joint Autoregressive and Hierarchical Priors for Learned Image Compression". In: *NeurIPS, Montréal, Canada*. 2018, pp. 10794–10803.

[20] Anish Mittal et al. "No-reference image quality assessment in the spatial domain". In: *IEEE Transactions on image processing* 21.12 (2012), pp. 4695–4708.

[21] Brendan Reidy et al. "HiRISE: High-Resolution Image Scaling for Edge ML via In-Sensor Compression and Selective ROI". In: *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 2024, pp. 1–6.

[22] Dongdong Tang et al. "p LPAQ: Accelerating LPAQ Compression on FPGA". In: *2022 International Conference on Field-Programmable Technology (ICFPT)*. IEEE. 2022, pp. 1–6.

[23] Dongdong Tang et al. "Stem: Streaming-based fpga acceleration for large-scale compactions in lsm kv". In: *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE. 2024, pp. 3893–3905.

[24] *Tegrastats Utility*. https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvvib_docs/DRIVE_OS_Linux_SDK_Development_Guide/Utilities/util_tegrastats.html. 2023.

[25] *The Future of Computing is Distributed*. [Accessed on October 14, 2023]. 2020. URL: https://www.datanami.com/2020/02/26/the-futureof-computing-is-distributed/.

[26] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[27] Workshop et al. *Challenge on Learned Image Compression*. https://compression.cc/. 2022.

[28] Li Xie et al. "Gaussian Rate-Distortion-Perception Coding and Entropy-Constrained Scalar Quantization". In: *arXiv preprint arXiv:2409.02388* (2024).

[29] Li Xie et al. "Output-constrained lossy source coding with application to rate-distortion-perception theory". In: *IEEE Transactions on Communications* (2024).

[30] Guanghao Yin et al. "Online Streaming Video Super-Resolution with Convolutional Look-Up Table". In: *arXiv preprint arXiv:2303.00334* (2023).

[31] Jingyu Zhang et al. "Research on the application of computer vision based on deep learning in autonomous driving technology". In: *International conference on wireless communications networking and applications*. Springer. 2023, pp. 82–91.

[32] Richard Zhang et al. "The unreasonable effectiveness of deep features as a perceptual metric". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 586–595.